PIMLex: A High-Performance Learned Index with Processing-in-Memory

Lixiao Cui, Kedi Yang, Yusen Li, Gang Wang, and Xiaoguang Liu, *College of Computer Science, Nankai University*

https://www.usenix.org/conference/fast25/presentation/cui

This paper is included in the Proceedings of the 23rd USENIX Conference on File and Storage Technologies.

February 25–27, 2025 • Santa Clara, CA, USA

ISBN 978-1-939133-45-8

Open access to the Proceedings of the 23rd USENIX Conference on File and Storage Technologies is sponsored by

PIMLex: A High-Performance Learned Index with Processing-in-Memory

Lixiao Cui, Kedi Yang, Yusen Li, Gang Wang *, and Xiaoguang Liu *
College of Computer Science, Nankai University

Abstract

The index structures represented by the learned indexes are crucial components of storage systems. However, their performance is restricted by the memory bandwidth/latency wall in conventional computer architectures. Processing-in-memory (PIM) technology is a promising solution by integrating processing units directly into memory devices. In this paper, we propose PIMLex, a well-designed learned index with PIM, to alleviate the memory-bound issue. PIMLex overcomes the capacity limitations of existing PIM hardware by employing a decoupled two-layer structure. This design simultaneously leverages the powerful data processing capabilities of PIM and the large capacity of conventional DRAM. Additionally, a PIM-friendly model structure is incorporated to minimize computational tasks that PIM struggles with. Combined with a hotness-aware replication mechanism that ensures load balancing across numerous PIM modules, PIMLex is able to deliver high performance across various workload patterns. We implement PIMLex on UPMEM, an available commercial PIM. PIMLex achieves 36.5× higher throughput than the PIM-based learned index baseline and 2.2× higher than the DRAM-based ALEX.

1 Introduction

The exponential growth of data processing demands in recent years place an increasing burden on conventional computer architectures. The traditional von Neumann architecture, which separates memory and processing units, struggles to keep pace with the throughput requirements of emerging data-intensive applications due to the mismatch between memory speed and CPU speed (i.e., "memory wall"). Processing-in-Memory (PIM) technologies [16,22,34,37,48,52] offer a promising approach to the "memory wall" issue. They integrate processing units directly within memory devices, enabling data-intensive applications to perform computations near the data, thus reducing the need for costly data movements between memory modules and the host CPU. Many prior works explored the design space under the PIM architecture, including graph processing [1, 25, 49, 64, 67], neural network [3, 15, 23, 30, 42] and database [5, 6, 26, 43, 44].

Index structures are the backbone components in various storage systems, including file systems and key-value stores. Recent studies propose replacing traditional ordered indexes with emerging learned indexes [14, 17, 18, 32, 39, 41, 56, 59], which utilize simple machine learning (ML) models to adapt to the data distribution and make accurate predictions about key locations, demonstrating significant performance advantages over traditional indexes [47, 58]. However, the memory wall still presents a significant performance bottleneck for learned indexes. The data lookup within the model prediction error bound [18, 32] and accesses to the model itself involve a lot of data movements between the host CPU and memory. Observations reveal that state-of-the-art learned indexes spend more than 60% of their execution time on memory stalls (Figure 2). As a typical data-intensive application, learned indexes are well-suited for PIM architectures. In this paper, we focus on designing a high-performance learned index based on PIM to overcome the memory-bound limitations. However, when current learned indexes are integrated into PIM architectures, several non-trivial challenges arise.

1) The conflict between the large space required by learned indexes and the limited capacity of commercial PIM devices. In modern in-memory systems, indexes typically consume around 55% of the total memory [63]. Moreover, previous studies [55, 58] have shown that some learned indexes result in significant space amplification. For instance, LIPP [59] can occupy over 5 times the space of the actual stored data [58]. Typically, UPMEM [10], a general-purpose PIM, offers a capacity of only 8GB per DIMM, which is significantly smaller than the capacity of conventional DRAM which can reach 64GB per DIMM. There is a conflict arising from the limited capacity of commercial PIM devices, as they are unable to fulfill the extensive space requirements of learned indexes, especially when dealing with large datasets.

2) The mismatch between the model structure of learned

*Corresponding Authors

indexes and the features of PIM devices. Due to the limitations of integrated circuit manufacturing processes, current available PIM devices can hardly incorporate powerful compute units [16, 38]. Even worse, the processors of UPMEM [10, 16] lack native hardware support for floating point operations. Instead, they emulate these operations using hardware-supported integer operations. However, learned indexes rely on model predictions for data processing, which involve multiple floating-point operations. Therefore, the overall performance of learned indexes is significantly hindered by the limited computing performance of PIM.

3) The load imbalance issues caused by skewed access patterns in distributed PIM architecture. A typical PIM-enabled platform consists of a set of independent PIM modules [10, 16, 26]. To fully utilize the PIM resources, prior PIM-based ordered indexes [9, 44] often divide indexes into several parts (e.g., through range partitioning of the key space), and store them across M PIM modules. However, scenarios involving learned indexes often exhibit skewed workloads [7, 8, 53, 61], where a small portion of items receive frequent accesses. In the case of learned indexes in a distributed PIM architecture, these skewed accesses can result in some PIM modules being heavily loaded while others remain idle. Such load imbalance significantly degrades performance.

In this paper, we propose PIMLex, a well-designed learned index with PIM, to address the aforementioned challenges.

• To tackle challenge 1, PIMLex uses a decoupled structure comprising an in-PIM search layer and an in-DRAM data layer. The data layer, housed in conventional DRAM, manages the complete set of keys and values. The search layer only contains anchor keys sampled from the data layer and builds models based on these sampled keys. During an index operation, the search layer conducts the majority of memory accesses to identify the approximate position of the target key within the data layer. Then, in the data layer, only a few memory accesses are required to locate the target key and manipulate corresponding data. This structure transfers the bulk of memory accesses to the efficient PIM modules, while minimizing PIM space usage, enabling scalable capacity and high performance.

• To address challenge 2, we optimize the model structure by adhering to the principle of "trading more memory access for less computation" to align with the features of PIM. Unlike previous learned indexes that used hierarchical multi-level models (each level involving one computation), PIMLex only selectively employs model predictions for certain model levels. Additionally, during model computations, PIMLex employs lookup tables that store pre-computation results to reduce floating-point operations. These methods augment memory accesses to the model while substantially reducing computational overhead, leveraging the excellent memory access capabilities of PIM.

• To solve challenge 3, we propose a hotness aware replication mechanism for the search layer to maintain load balance when handling skewed workloads. The search layer is partitioned according to key ranges and distributed across different PIM modules. PIMLex creates a varying number of replicas for different partitions based on their access popularity. With multiple replicas, search layer partitions with higher access rates can distribute requests to more PIM modules, alleviating the burden on heavily loaded modules.

We implement a prototype of PIMLex and evaluate its performance on UPMEM [10, 16], a commercially available PIM hardware. Our experiments show that PIMLex outperforms the PIM-based learned index baseline by 36.5× and outperforms the DRAM-based ALEX [17] by 2.2×. PIMLex serves as a preliminary step for learned indexes on PIM, showcasing the potential of PIM for indexes. As the capabilities (e.g., computation) of future PIM devices improve, PIM-based learned indexes are expected to achieve even better performance.

2 Background and Motivation

2.1 PIM Architecture

Many PIM architectures [16, 20, 28, 29, 34, 35, 37, 52] have been proposed recently, including logic gates embedded in each memory cell [29] and utilizing parallel processors that are placed near memory [16, 33, 37]. These PIM devices exhibit extremely high memory bandwidth due to the ability to process near data. For example, DRAM-based UPMEM [16] has a peak bandwidth of 80GB/s per DIMM [24]. For HBM-PIM using more advanced HBM technology, its peak bandwidth can reach 2TB/s per cube [34]. The bandwidth of PIM is generally more than 8x that of conventional DRAM, thus facilitating the processing of various data-intensive applications [6, 26, 43, 44].

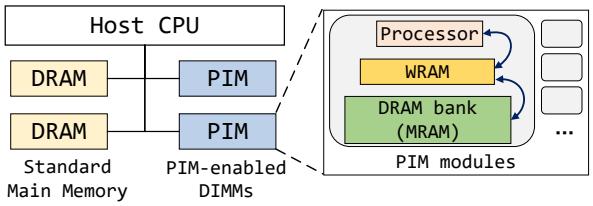
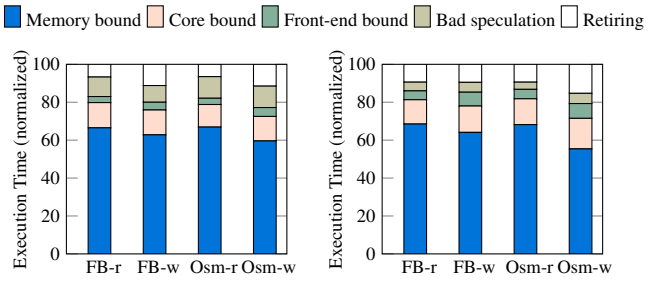


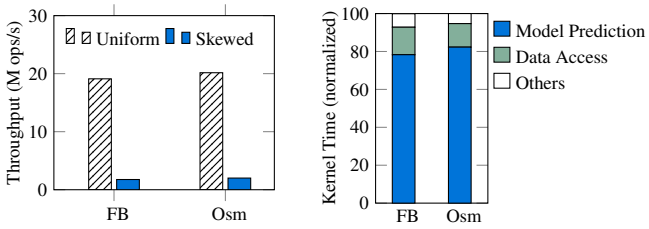
Figure 1: The structure of the UPMEM PIM system.

Figure 1 depicts the architecture of a generic PIM-enabled system with UPMEM [10, 11, 16]. The PIM devices are connected to CPU via DIMM slots. The standard DRAM DIMMs without data processing capabilities are also equipped. A PIM DIMM comprises several PIM modules, with each module consisting of a memory bank and a general-purpose on-bank processor. These PIM modules form a shared-nothing distributed architecture, where each processor can only manipulate data within its local memory bank. The right side of Figure 1 illustrates the internal architecture of a PIM module. The processor, known as the DRAM Processing Unit (DPU), is a 32-bit in-order RISC core. Due to hardware and



(a) ALEX. (b) LIPP.

Figure 2: Execution time breakdown for learned index operations. *r* represents *Get* and *w* represents *Insert*.



(a) Get Throughput. (b) Breakdown.

Figure 3: Performance and execution time breakdown of *BasicLex*.

cost limitations, the DPU only provides native integer ALUs. Floating point operations, multiplication and division operations are emulated through software, leading to orders of magnitude slower performance than integer addition and subtraction. Each PIM module comprises a 64KB fast memory area, called Working RAM (WRAM), which is based on SRAM technology. The DPU cannot directly access its memory bank, namely Main RAM (MRAM). It needs to first load data from MRAM to WRAM, and then perform data operations. MRAM has a much larger capacity than WRAM (64MB), but its access performance is inferior.

PIM programming follows an accelerator model where programmers need to prepare an execution program (kernel function) in advance and load it onto PIM modules. The execution procedure involves four steps. First, the host CPU prepares a buffer containing a batch of tasks for each PIM module. Then, the host CPU transfers these task buffers to PIM modules. Next, CPU launches PIM kernel functions, which triggers PIM modules to execute their assigned tasks and obtain the results. Finally, the host CPU gathers the running results by moving them to main memory. This batch execution process has been widely used by previous applications with PIM [26, 27].

2.2 Learned Index

Learned indexes [17, 18, 32, 39, 41, 56, 59] demonstrate significant advantages over traditional tree-based indexes by exploiting easy-to-use models. Specifically, learned indexes usually use linear regression models to fit the cumulative distribution function (CDF) of sorted data. Therefore, the process of searching data can be guided by the prediction of models, which enables efficient index operations. To accurately fit the data distribution, learned indexes usually adopt hierarchical models [17, 18, 32, 41], which involve multiple calculations for each index operation.

When processing queries, learned indexes first obtain the predicted positions through the hierarchical model, which involves multiple model searches and calculations. Then, they find the exact location of the requested key by "last mile" searching, i.e., some additional searches around the predicted position. Some indexes, like PGM [18] and ALEX [17], store sorted keys in contiguous arrays. They conduct a binary search or an exponential search starting from the predicted location to find the requested key. Some indexes, like LIPP [59], adopt a chain scheme that creates new nodes to store data. They need to traverse other nodes via pointer chasing from the predicted location. The model searches and the "last mile" search present a performance bottleneck due to numerous costly memory accesses. We analyze two prominent learned indexes, ALEX [17] and LIPP [59], from a micro-architectural perspective using Intel VTune Profiler. We use two datasets, *FB*[1] and *Osm*[2]. Figure 2 illustrates that for both *Get* and *Insert* operations, the majority of stalls are caused by memory bound. These phenomena are also confirmed by previous study [2]. These observations motivate us to address the memory constraints of learned indexes.

2.3 Learned Index on PIM

The powerful memory capability of PIM makes it suitable for optimizing learned indexes. To explore the performance of learned indexes on PIM, we develop a prototype learned index (represented by *BasicLex*) with UPMEM. Following prior work [9, 44], we partition the key-value pairs evenly into *M* parts using range partitioning and assign each part to a PIM module. Within each PIM module, we use the hierarchical optimizing piecewise linear regression (PLR) models proposed by PGM [18] to construct the learned index. These PLR models, with a specified error bound ε, generate predicted positions that fall within a 2ε range of the actual data. We set ε to 32, which yields good performance under traditional architecture. Both models and key-value pairs are stored in MRAM region. Our evaluation involves 512 PIM modules and uses *FB* and *Osm* datasets.

For the overall search throughput, under uniform access mode, *BasicLex* demonstrates favorable performance. However, in the case of skewed access mode, *BasicLex* experiences significant performance degradation due to load imbalance. Some PIM modules are busy while others are idle. In extreme

[1]The randomly sampled Facebook user IDs.
[2]The randomly sampled cell ids on OpenStreetMap.

cases, only one PIM module will process the queries. As Figure 3(a) shows, when using the Zipfian distribution [12] with a skewness of 0.99, *BasicLex* achieves only 9.2% to 9.9% of the throughput compared to the uniform distribution. These observations motivate us to develop an efficient load-balancing solution for PIM-based learned indexes.

Figure 3(b) shows the breakdown of kernel time (i.e., PIM execution time) for *BasicLex*. About 80% of the time is dedicated to model prediction, of which more than 50% is computational overhead. *BasicLex* constructs multi-level models on PIM. For example, for *FB* dataset, the number of model levels ranges from 3 to 4 depending on the complexity of the data across different PIM modules. Consequently, each lookup necessitates 3 to 4 floating-point multiplication operations. However, the computational capabilities of PIM are limited. With a single UPMEM PIM module, the throughput for floating-point multiplication is only 1.91 MOPS [24]. This leads to significant computational overhead for model predictions. In contrast, the memory bandwidth of a single PIM module can achieve between 630 MB/s (MRAM) and 2800 MB/s (WRAM) [24]. These motivate us to optimize the model structures by considering the features of PIM.

3 PIMLex Design

3.1 Overview

We introduce PIMLex, a learned index designed to mitigate the memory-bound issue by leveraging PIM. Following prior studies [26, 27], PIMLex adopts a batch processing model for operations. The challenges of PIMLex are how to overcome the limitations of existing PIM hardware, encompassing limited capacity, architecture that is not conducive to models, and load imbalance under skewed workloads. To this end, we propose a decoupled index structure, a PIM-friendly model structure and a hotness aware replication mechanism.

The overview of PIMLex is shown in Figure 4. In general, PIMLex employs a decoupled structure comprising a search layer on the PIM side and a data layer on the DRAM side (i.e., standard main memory). The search layer consists of anchor keys that are obtained by sampling the primary data array, which we will discuss in detail later. Through range partitioning, the search layer is divided into small partitions and distributed across multiple PIM modules. Within each partition, a series of models are created based on anchor keys to enhance search efficiency. The model structure of the search layer is specifically designed to align with the characteristics of PIM (i.e., powerful memory capability but weak computation capability), as explained in Section 3.2, to optimize execution efficiency on the PIM side. Additionally, to ensure load balancing, the frequently accessed partitions are assigned to more PIM modules, forming multiple replicas. We will provide further details on the hotness-aware replication mechanism in Section 3.3. The data layer consists of a

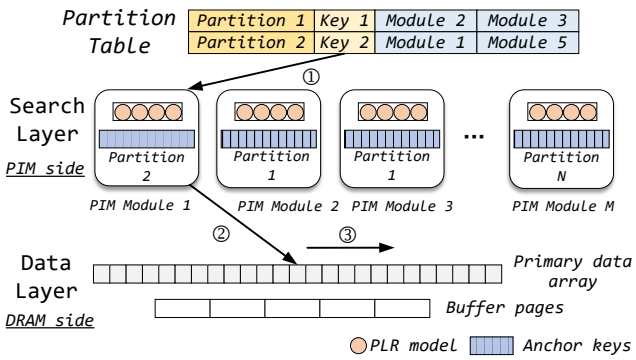


Figure 4: Overview of PIMLex.

primary data array and several buffer pages. The primary data array stores key-value pairs in order, enabling it to handle reads and value updates. The buffer pages are used for handling the insertion of new data. The data in the buffer page is also ordered for easy lookup. We will explain the index operation processing in Section 3.4.

As shown on the upper side of Figure 4, PIMLex maintains a partition table, which keeps track of the PIM modules where each partition of the search layer is located. To achieve load balancing, some partitions have replicas in multiple PIM modules. For example, partition 2 has replicas in module 1 and module 5. The partition table also retains the minimum key of each partition, enabling PIMLex to determine the corresponding partition for a given key. Regardless of the operation type (*Get*, *Insert*, *Update* and *Range query*), PIMLex follows a three-step execution procedure as depicted in Figure 4. In step ①, PIMLex checks the partition table to identify the PIM module to which the requested key K_t belongs. If the corresponding partition has multiple replicas, PIMLex randomly selects one of these replica modules. For example, PIMLex selects module 1 from the replicas of partition 2 as Figure 4 shows. In step ②, PIMLex performs a lookup on the search layer in the selected PIM module, obtaining the approximate position POS_{approx} of K_t in the primary data array. Finally, in step ③, PIMLex utilizes POS_{approx} to locate the actual position of K_t within the data layer and performs reading and writing on the primary data array and buffer pages.

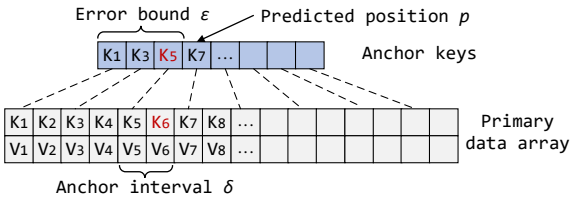


Figure 5: The relationship between anchor keys and primary data array.

The decoupled structure of PIMLex can utilize PIM to handle most memory accesses. As Figure 5 illustrates, PIMLex selects anchor keys at intervals of δ from the primary

data array. Then PIMLex trains models with an error bound ε based on these anchor keys. Suppose we want to find K_6. PIMLex searches for the largest anchor key less than K_6 within the prediction range $[p-\varepsilon, p+\varepsilon]$ (i.e., the red K_5). Since the anchor keys are partitioned into multiple PIM modules, the position of K_5 in the anchor keys (i.e., P_{K5}) can be obtained by adding the position of K_5 within its partition (assuming it belongs to *partition 1*) and the offset of *partition 1* in all anchor keys. Consequently, the starting search position of the requested key in the primary data array is $P_{K5} \times \delta$. PIMLex then performs a search within the range $[P_{K5} \times \delta, P_{K5} \times \delta + \delta]$ in the primary data array to locate K_6. Assuming binary search is employed, the search layer requires $log(2\varepsilon)$ memory accesses for anchor keys and some memory accesses to access the models. Accessing the primary data array only requires $log(\delta)$ memory accesses. We set the default δ to 8 so that PIMLex can complete the search in the primary data array within a cacheline[3].

The decoupled two-layer design of PIMLex leads to a smaller search layer size. The number of anchor keys is only $1/\delta$ of the primary keys. Besides, since the search layer's role is solely to identify the approximate data location and does not involve reading or writing data, there is no need to store values associated with the anchor keys. This capability enables PIMLex to handle large data volumes using small-capacity PIM devices.

3.2 PIM-friendly Model Structure

PIMLex constructs machine learning models to enhance search efficiency within the search layer. A significant challenge arises in adapting the model structure to align with the hardware characteristics of PIM, which feature robust memory capability but limited computing capability. To this end, we optimize the model structure based on the principle of "trading more memory access for less computation".

Model Search Method Selection. Prior indexes such as ALEX [17] and PGM [18] typically apply multi-level models. The left side of Figure 6 shows the multi-level models of PGM [18]. Each linear model is trained based on the keys of a segment. A model contains the first key of a segment (k), the model's slope (A) and intercept (B). These models predict the position of a key within its segment while maintaining a prediction error bound ε. Training of high-level models is conducted in a bottom-up manner using the lower-level models as input, until the number of top-level models is reduced to a manageable quantity. Search operations at each level are based on the upper-level models (**model-based search**). Initially, a prediction is made based on the upper-level model with a cost of $Cost_{prediction}$[4]. Subsequently, a binary search is performed within a range of 2ε around the predicted position to locate the desired model, incurring a cost of

[3]The size of a key is 8-byte, the same as prior work [17, 58].

[4]Primarily the floating-point calculation cost.

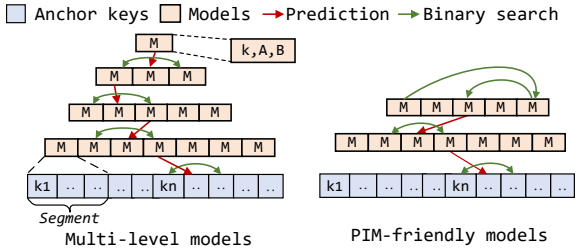


Figure 6: Traditional multi-level models versus PIM-friendly models using global search for upper levels.

$log(2\varepsilon) * Cost_{search}$. While this multi-level model structure is efficient with a host CPU due to reduced memory access for model search, it performs poorly under the PIM architecture with limited computing capability.

Another search method is **global binary search**. It directly performs binary searching using the k values of all models within that level, incurring a cost of $log(Num_{models}) * Cost_{search}$ (Num_{models} denotes the number of models in the level). This method does not require predictions from the upper-level model, mitigating the computational overhead. By comparing the costs of employing **global binary search** versus **model-based search**, PIMLex can opt for the scheme with the lower cost. The costs associated with memory access ($Cost_{search}$) and computation ($Cost_{prediction}$) can be conveniently quantified [24]. In our implementation, PIMLex gives precedence to apply model-based search for the levels below until the global binary search demonstrates a lower cost in a specific level as the right side of Figure 6 shows. Compared to the original multi-layer model, our approach introduces global binary search, which reduces model computation by increasing memory access. This design choice is particularly well-suited for PIM systems, given their robust memory access capabilities.

Lookup-table based Model. Despite the reduction in overall computational overhead achieved by the method described above, the calculation of a single linear model still involves floating-point computations. To eliminate the need for floating-point operations, we propose the utilization of lookup-table based models. In the context of learned indexes, the predicted position Pos_{full} is determined by the multiplication of the search key with the slope A of the linear model. This multiplication operation on the search key can be decomposed into separate operations on its high and low bits. We can approximate the true prediction position Pos_{full} of a full search key through the calculation results of its own high bits (Pos_{lower}) and the neighboring high bits (Pos_{upper}). As Figure 7 shows, for key `0xff11`, its corresponding high bits are `0xff10` and `0xff20`, with $p1$ and $p2$ representing its estimated prediction value.

By pre-calculating the results of high-bits multiplication within the key range and storing them in a lookup table, we can eliminate floating-point operations during model calculations. Instead, accessing the lookup table provides the corre-

sponding results. For instance, Figure 7 shows a lookup table with 4 slots that stores the operation results of the high 12 bits of the keys. However, the use of a lookup table poses two challenges. Firstly, the table may consume a significant amount of memory space. The accuracy of the obtained results improves with an increased number of slots in the table. To strike a balance between space and accuracy, we constrain the number of slots in the lookup table to S (default value of 16). Secondly, the prediction positions derived from the lookup table may lack precision. Therefore, a search method based on these approximate positions must be considered. In the case of the floating-point based model, the desired key falls within the search range $[Pos_{full} - \varepsilon, Pos_{full} + \varepsilon]$. Utilizing the lookup table, we obtain the approximate prediction positions, Pos_{lower} and Pos_{upper}, where $Pos_{lower} \leq Pos_{full} \leq Pos_{upper}$. Consequently, a binary search within the range $[Pos_{lower} - \varepsilon, Pos_{upper} + \varepsilon]$ ensures the identification of the desired result.

Due to the imprecise prediction positions of the lookup-table based model, it results in higher data search overhead compared to the floating-point based model. Therefore, PIMLex selects the most efficient model type by compare the overall cost of different model types. For the floating-point model, the total cost is $Cost_{prediction} + Cost_{search} * log(2\varepsilon)$. In contrast, for the lookup-table based model, the total cost mostly involves the searching within the error range, that is $log(Pos_{upper} - Pos_{lower} + 2\varepsilon) * Cost_{search}$. Additionally, a single memory access is also required for table lookup, incurring a cost of $Cost_{search}$. As the pairs of Pos_{upper} and Pos_{lower} vary for different request keys, we select the maximum value of $Pos_{upper} - Pos_{lower}$ in the lookup-table based model to compute the overall cost, representing the worst-case scenario.

Placing Models in WRAM. Within the PIM module, the memory is divided into a fast WRAM area and a slow MRAM area. To accelerate model search, PIMLex stores models in the fast WRAM and places the anchor keys in the MRAM. However, the capacity of WRAM is small (64KB). When using a small ε, a large number of models are generated, exceeding the capacity of the small WRAM. To this end, PIMLex will adjust the value of ε. If the number of generated models based on the current ε_{cur} is too large to fit in WRAM, we adjust ε_{cur} to $2 \times \varepsilon_{cur}$, until the generated models can be placed into the WRAM.

3.3 Hotness Aware Replication Mechanism

When dealing with skewed workloads, learned indexes often face the challenge of maintaining load balance across PIM modules. To address this challenge, we propose creating additional replicas for frequently accessed hot data to ensure even distribution of access requests. The decoupled structure of PIMLex helps mitigate the difficulties associated with the replication mechanism. The anchor keys in the search layer are immutable (except during model retraining), which

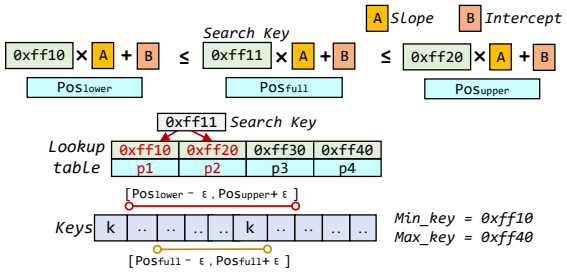


Figure 7: Lookup-table based model.

means the replicas are read-only and PIMLex does not need to resolve replica consistency issues on updates.

We adopt partition granularity for identifying hotness and creating replicas in PIMLex. The search layer is divided into N disjoint partitions based on key ranges. These N partitions are placed across M PIM modules ($M \geq N$) and each PIM module holds exactly one partition. The hot partitions occupy more PIM modules to accommodate their replicas, while the cold partitions are assigned to only a few PIM modules. To achieve optimal performance, our hotness aware replication mechanism needs to solve two problems. The first is how to determine the appropriate number of replicas for each partition to achieve load balancing. The second is how to maintain performance during changes in hotspot distribution.

Load Balancing Goal. Assuming that each partition needs to process a specific number of tasks denoted as T_1, T_2, ..., T_N, and PIMLex creates a certain number of replicas for each partition denoted as R_1, R_2, ..., R_N. These values are integers and satisfy the conditions in Equation 1. R_i is at least 1, that is, a search layer partition must be stored in at least one PIM module to enable the search layer operations on the PIM side.

$$\begin{aligned} R_1 + R_2 + ... + R_n &= M \\ 0 < R_1, R_2, ..., R_n &\leq M \end{aligned} \tag{1}$$

We define the load factor L_i as $(T_i/R_i)/(T_{total}/M)$, which represents how busy partition i is when using R_i replicas (PIM modules). The closer L_i is to 1, the less busy it is. In current architecture, multiple PIM modules are executed in parallel. The total execution time of PIM is determined by the busiest partition. Thus, we define global load factor L_{global} as the busyness degree of the busiest partition among all partitions.

$$L_{global} = \max(L_i) \quad i = 1, 2, ..., N \tag{2}$$

The goal of load balancing is to configure the values of R_i that minimize L_{global}. When L_{global} is equal to 1, all PIM modules have the same number of tasks to process.

Two-stage Load Balancing Algorithm. To efficiently find the minimum value of L_{global}, we propose a two-stage algorithm. This algorithm determines the number of replicas R_i for each partition and effectively achieves a sub-optimal load balancing goal. The algorithm details are shown in Algorithm 1. In the first stage, we estimate the number of replicas (PIM modules) for each partition. Initially, we calculate the

Algorithm 1 *Two-stage Load Balancing Algorithm*

```
1:  /* First stage, rough estimation */
2:  T_avg ← T_total / M
3:  i ← 0
4:  while i < N do
5:      R_i ← max(⌈T_i / T_avg⌉, 1)
6:      L_i ← (T_i / R_i) / (T_total / M)
7:      i ← i + 1
8:  end while
9:  R_total ← sum(R_1, R_2, ..., R_N)
10: if R_total != M then
11:     Increase or decrease some R_i values so that R_total == M
12: end if
13: /* Second stage, fine-tuning */
14: Original_L_global ← max(L_i)
15: k ← 0
16: while k < Num_fine_tune do
17:     R_busy ← busiest_partition()
18:     R_idle ← idlest_partition()
19:     R_idle ← R_idle − 1
20:     R_busy ← R_busy + 1
21:     Current_L_global ← max(L_i)
22:     if Current_L_global < Original_L_global then
23:         Original_L_global = Current_L_global
24:     else
25:         Restore the original R_busy and R_idle and return
26:     end if
27:     k ← k + 1
28: end while
```

average number of tasks T_{avg} that each PIM module should handle (line 2). Then, by comparing the number of tasks each partition undertakes T_i with T_{avg}, we can obtain a rough estimate of the number of replicas R_i for each partition (lines 4 to 8). Meanwhile, we calculate the load factor L_i under current R_i. The total number of replicas R_{total} obtained through this method may not equal the number of PIM modules M. Therefore, in lines 10 to 12, we adjust some R_i values to make R_{total} consistent with M. If R_{total} is greater than M, we decrease the R_i of the partition with the minimum load factor until R_{total} meets the conditions, and vice versa.

The first stage provides an initial set of R_i values, but it may still result in a relatively large L_{global}. To further reduce L_{global}, we proceed with fine-tuning in the second stage. The fine-tuning process consists of Num_fine_tune rounds (line 16). In each round, we obtain the number of replicas of the busiest and idlest partitions, R_{busy} and R_{idle} respectively (lines 17 and 18). Then in lines 19 and 20, we try to reassign a PIM module from the idle partition to the busy partition (i.e., changing R_{idle} and R_{busy}). If the resulting L_{global} after fine-tuning is smaller than the previous value, we accept this adjustment. Otherwise, we revert to the state before this adjustment (lines 21 to 25). The fine-tuning stage allows us to approach the optimal L_{global} step by step. We can use Num_fine_tune to weigh the cost of the algorithm and the level of optimization achieved for L_{global}.

Hotness Identification and Replication Adjustment. The prerequisite for determining the number of replicas R_i is to obtain the number of tasks T_i that each partition needs to process. Since we cannot get the future T_i values, we rely on the number of tasks assigned to each partition in the current batch as a predicted substitute for T_i. These values can be easily obtained through the partition table. Because access hotspots are relatively stable over a period of time [4, 8, 53], the hotness distribution of the current batch can efficiently represent the situation in future batches.

When there are changes in hotness distribution [8, 61], PIMLex adjusts the number of replicas to handle dynamic workloads. After creating replicas based on the current hotness, we record the current L_{global} as L_{global_old}. When executing future each batch, we calculate the L_{global_cur} at that time. If L_{global_cur} is greater than $\eta \times L_{global\_old}$ (η is a configurable threshold, with a default value of 1.5), it indicates a change in hotness. In such cases, PIMLex determines a new replication scheme that aligns with the current hotness using the two-stage load balancing algorithm. Subsequently, the search layer partitions are transferred to different PIM modules based on the newly determined R_i. It should be noted that the search layer maintains a duplicate copy in DRAM, meaning that data transfer only occurs from DRAM to PIM.

Although the above replication adjustment method achieves effective load balancing, it needs to transfer the search layer partitions to all M PIM modules whenever there is a change in hotness. This transmission process can be time-consuming, especially as the size of the search layer increases. To address this issue, we propose a quick replication adjustment method. This quick method only executes the second stage of the two-stage load balancing algorithm, meaning that it changes replicas for at most Num_fine_tune PIM modules, while leaving the partitions stored in other modules unchanged. At this time, data only needs to be transferred to the Num_fine_tune PIM modules. The quick adjustment method compromises load balancing capabilities, but offers faster adjustment speed, making it more suitable for highly dynamic workloads.

During replication adjustment, PIMLex can still perform index operations through the duplicate copy of the search layer in DRAM. However, this leads to increased contention among CPUs. Therefore, we provide an option to either enable or block index operations while adjusting replicas.

3.4 PIMLex Operations

As Section 3.1 describes, the operations of PIMLex has three steps: partition table checking (step ①), the execution of search layer (step ②) on PIM side, and data layer execution ③) on DRAM side. We consider the situation of running multiple batches on PIMLex. A pipeline approach is adopted to improve overall throughput. It overlaps the execution on the PIM side (PIM execution and DRAM-PIM data transfer) with the DRAM side (DRAM execution). Below we introduce the execution process of PIMLex operations.

Insert. This operation inserts new key-value pairs into PIMLex. PIMLex uses buffer pages to handle new data insertion. Each anchor key is associated with a buffer page. Figure 8 illustrates the process of handling key insertions. The first step is to locate the corresponding buffer page. PIMLex achieves

this by finding the largest anchor key that is not greater than the key being inserted. This search operation is performed by the search layer, leveraging the benefits of PIM. For example, when inserting K_6, the position of K_5(the third one) is identified among the anchor keys, indicating that the data should be inserted into the third buffer page. The second step is to insert data to the found buffer page. Each buffer page is a sorted array that can hold up to Max_{kv} key-value pairs. We set Max_{kv} to 8 by default. To guarantee the correctness of concurrent insertion, each buffer page is protected by a lock. When processing skewed inserts, some buffer pages may become full and unable to accommodate new data. PIMLex builds a modified B+-tree as an overflow tree for each partition to manage this portion of data. The overflow tree also leverages PIM to speed up insertion. Its internal nodes are placed on the PIM side (i.e., the PIM modules where the partition replicas sit). When the buffer page is full during insertion, PIMLex will search the overflow tree on the PIM side to finds the corresponding leaf node on the DRAM side. Then it writes new data to the leaf nodes.

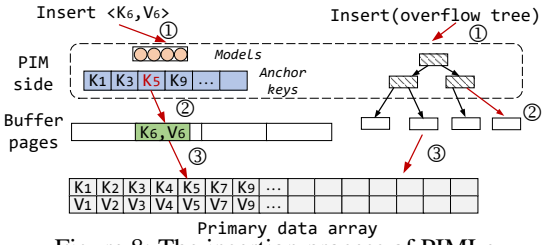


Figure 8: The insertion process of PIMLex.

In the third step, PIMLex performs data merging and rebuilds a new search layer when the data held by the buffer pages and overflow trees reaches a threshold (half of the primary array size by default). To begin, PIMLex creates a new primary array by merging buffer pages and overflow trees with the old primary data array. Although the data merging is a memory-bound operation, it is conducted entirely on the DRAM side. Since merging the buffer pages and primary array only requires a single scan, executing this operation on the PIM side would involve transmitting the buffer pages and primary array to the PIM modules, which is not efficient due to the low DRAM-PIM transmission bandwidth [24]. The merging process is performed in parallel by evenly distributing the primary data array, buffer pages and overflow trees to different threads based on the key range. Next, PIMLex creates a new search layer. The construction of anchor keys is also executed in parallel. The computationally intensive task of model training is carried out on powerful host CPUs, similar to prior work [18], utilizing multiple threads for efficient training. Once the new search layer is ready, PIMLex employs the hotness aware replication mechanism to determine the number of replicas for different partitions and loads the search layer partitions onto the PIM modules. Finally, the partition table is updated to align with the new search layer.

Get and Update. PIMLex first finds the approximate location through the search layer, then it looks for the given key in primary data array as mentioned in Section 3.1. If the given key is not found in the array, PIMLex searches the buffer pages and overflow tree in sequence. For mixed *Get-Update* operations, PIMLex applies read-write locks on primary data array. By default, every 512 consecutive key-value pairs share a lock. The buffer pages and overflow trees, on the other hand, are protected by their respective page/node locks.

Range Query. The range query operation (a.k.a. *Scan*) returns a series of key-value pairs within a given key range $[K_{first}, K_{last}]$. PIMLex initially finds the first position greater than or equal to K_{first} in the primary data array, buffer pages and overflow tree respectively. This operation is similar to the *Get* process and also uses the search layer on the PIM modules. Then, PIMLex scans the primary data array, buffer pages and overflow tree in reverse from the starting position until the scanned key exceeds the requested key range. If the overflow tree contains data, PIMLex also performs a range query on it.

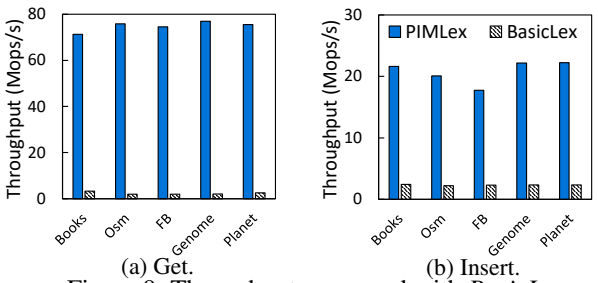
4 Evaluation

4.1 Experiment Setup

We implement PIMLex with UPMEM [10, 11, 16]. The evaluation platform is a Linux server with two Intel(R) Xeon(R) Silver 4110 CPUs (16 logical cores, 2.10GHz, 11 MB L3 Cache). For memory devices, the server is equipped with 4 conventional DRAM DIMMs (32GB per DIMM) and 4 UPMEM DIMMs. Each DIMM uses a memory channel. Each UPMEM DIMM consists of 128 PIM modules. Thus, we have a total of 512 PIM modules (referred to as M in Section 3.3) that we use for evaluating PIMLex. A PIM module has a 64MB MRAM area and a 64KB WRAM area. The capacity of a UPMEM DIMM is 8GB, considering only the MRAM capacity. In our evaluation, PIMLex runs with 32 threads to utilize all the host CPU cores. The number of search layer partitions is 128 (i.e., N in Section 3.3).

Compared Counterparts. We implemented a basic PIM-based learned index (*BasicLex*) as our baseline. BasicLex places the entire index on the PIM side and uses range partitioning to divide the index into different PIM modules. Unlike PIMLex, it does not employ any load balancing methods and still applies the original multi-level models.

We also compare PIMLex with some state-of-the-art DRAM-based learned indexes. (1) ALEX [17], which adopts adaptive hierarchical models for location prediction and gapped arrays for data placement. We use its concurrent version proposed in [58]. (2) LIPP [59]. It employs the fastest minimum conflict degree algorithm to improve prediction accuracy. We also utilize its concurrent version proposed in [58]. (3) FINEdex [39]. It incorporates fine-grained level-bins to handle inserted data. (4) SALI [21]. It uses a node-evolving strategy to adapt to varying workloads and a lightweight



(a) Get. (b) Insert.

Figure 9: Throughput compared with *BasicLex*.

concurrent approach to enhance scalability. Evaluating these DRAM-based indexes on the PIM-enabled server would lead to an unfair advantage for PIMLex. This is because PIMLex can leverage the memory bandwidth of 8 DIMMs (conventional DRAM and UPMEM), whereas DRAM-based learned indexes can only utilize 4 DRAM DIMMs. Therefore, we evaluate these DRAM-based indexes on a server equipped with 8 DRAM DIMMs (32GB per DIMM). This server features with two Intel(R) Xeon(R) Silver 4210 CPUs (20 logical cores, 2.20GHz, 13.75 MB L3 cache). Consistent with PIMLex, we run the DRAM-based counterparts with 32 threads.

Another PIM-based index, PIM-tree [26], is also compared. PIM-tree is an index combining a B+-tree and a skip-list and is designed to handle skewed workloads. It addresses load imbalance by fetching hot data to the DRAM side and handling hot requests with the host CPU. Because PIM-tree also utilizes both DRAM and PIM DIMMs, we evaluate it on the same platform with PIMLex.

Workloads. We utilize five real-world datasets: (1) *Books* [47], the book sales popularity from Amazon. (2) *Osm* [47], the randomly sampled cell IDs on OpenStreetMap. (3) *FB* [47], the randomly sampled Facebook user IDs. (4) *Genome* [58], the local pairs in human chromosomes. (5) *Planet* [58], the Planet IDs in OpenStreetMap. The *Books* and *Osm* each contain 800 million keys. The *FB*, *Genome* and *Planet* each contain 200 million keys. The key type for all five datasets is an 8-byte integer. In our evaluation, the value size is 32-byte by default.

Table 1: The total time (in seconds) of PIM execution and PIM-CPU communication under skewed workload. *Opt* refers to hotness aware replication mechanism, while *Baseline* indicates the absence of any load balancing methods.

	Books	Osm	FB	Genome	Planet
Opt	0.78	0.85	0.83	0.79	0.79
Baseline	12.47	12.85	12.55	12.57	12.51

4.2 Comparison with PIM-based Baseline

Basic Performance. We first compare the basic performance of PIMLex with BasicLex. We conduct *Get*-only and *Insert*-only workloads. For *Get*-only workload, we utilize the YCSB [12] benchmark to generate skewed workloads following the Zipfian distribution with a skewness of 0.99. For *Insert*-only workload, we refer to the hotspot mode in [55] to generate test data, where data are randomly sampled from a small portion of the dataset. In our evaluation, 95% of the inserted keys are sampled within 20% of the dataset. We first bulk load partial data into indexes before conducting evaluations. For *Get*-only workload, we bulk load 200 million keys. For *Insert*-only workload, we bulk load 100 million. The results are depicted in Figure 9.

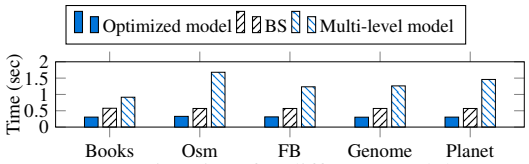
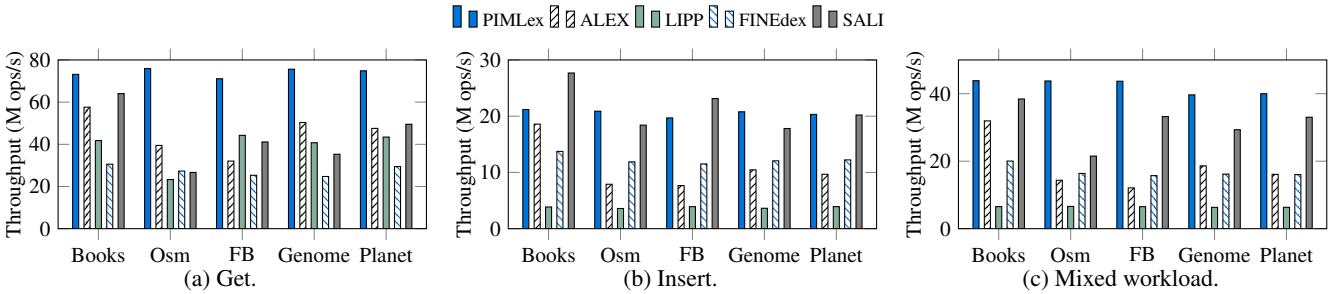


Figure 10: Execution time for different model structures.

It is observed that PIMLex has significant performance advantages over BasicLex in both *Get* and *Insert* workloads. The *Get* throughput of PIMLex exceeds that of BasicLex by up to 36.5×. For *Insert*, PIMLex outperforms BasicLex by up to 9.5×. Two reasons contribute to this performance gap. First, PIMLex employs a PIM-friendly model structure, which substantially reduces the cost of model prediction. Secondly, the load balancing mechanism of PIMLex can better handle skewed workloads, while BasicLex lacks robustness to query skew. These two points are discussed in detail below.

Effect of Model Structure. To validate the effectiveness of the model structure of PIMLex, we evaluate our PIM-friendly model structure (denoted as the optimized model) against two counterparts: (1) multi-level model, which is the original model structure used in existing learned indexes. (2) BS, which is the binary search method that directly searches anchor keys without using models. Their PIM execution times are shown in Figure 10. The original multi-level model performs the worst, even lower than BS, highlighting the limited computational ability of PIM. Our PIM-friendly model trades increased memory accesses for fewer floating-point operations, thereby achieving a shorter execution time compared to the multi-level model. By placing the models into WRAM, our PIM-friendly model further decreases model search time. Across all datasets, the optimized model exhibits the shortest execution time.

Effect of Load Balancing Method. Table 1 displays the time consumption of the PIM side under skewed workload. Without load balancing method, the time cost of PIM execution and CPU-PIM communication is very high. This is because the busiest PIM modules need to receive and process a large number of tasks, becoming a bottleneck for the index. In contrast, the hotness-aware replication mechanism of PIMLex can distribute hotspot queries to multiple PIM modules and process tasks in parallel, thereby significantly reducing time consumption on the PIM side.



(a) Get. (b) Insert. (c) Mixed workload.

Figure 11: Throughput compared with DRAM-based learned indexes.

4.3 Comparison with DRAM-based Indexes

Basic Performance. In this section, we compare the performance of PIMLex with existing DRAM-based learned indexes using 200-million scale datasets. We use GRE [58], a benchmark tool for learned indexes. GRE comprises three basic workloads: *Get*-only, *Insert*-only, and a mixed workload (50% *Get* and 50% *Insert*). These workloads follow Zipfian distributions same with YCSB [12]. For *Get*-only workload, we bulk load 200 million key-value pairs and conduct 100 million operations. For *Insert*-only and mixed workloads, we bulk load 100 million key-value pairs and conduct 100 million operations. The results are presented in Figure 11.

Under *Get*-only workload, PIMLex outperforms other counterparts. Compared to ALEX, LIPP, FINEdex and SALI, PIMLex achieves up to 2.2×, 3.25×, 3.05× and 2.84× higher throughput, respectively. These observations demonstrate the efficiency of utilizing PIM devices to accelerate learned indexes. For *insert* performance, PIMLex outperforms ALEX, LIPP and FINEdex across all datasets. However, in certain datasets (e.g., *Books*), PIMLex is inferior to SALI. This is because the data writes in the data layer are still handled on the DRAM side. To enhance insert performance, we could place the data layer on the PIM side. However, the limited capacity of current PIM devices restricts the feasibility of this approach. We expect that advances in the capacity of future PIM devices will lead to further improvements in PIMLex. For mixed workload, PIMLex is the best performer among all the indexes. It outperforms ALEX, LIPP, FINEdex and SALI by up to 3.6×, 6.7×, 2.73× and 2.03×, respectively.

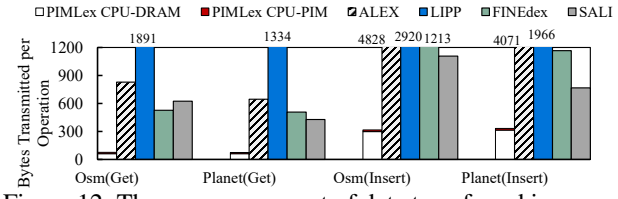


Figure 12: The average amount of data transferred in memory bus per operation.

CPU-Memory Communication. Figure 12 displays the communication volume between the CPU and memory devices for different index structures. By leveraging PIM for near-data processing, PIMLex necessitates fewer communications compared to DRAM-based learned indexes. The reduced communications of PIMLex liberate it from the constraints imposed by memory walls, enabling it to achieve superior performance. It is noteworthy that the data transfer bandwidth of CPU-PIM is significantly slower than CPU-DRAM, which has some impact on the performance of PIMLex. We expect that the future PIM architecture can enhance the CPU-PIM bandwidth.

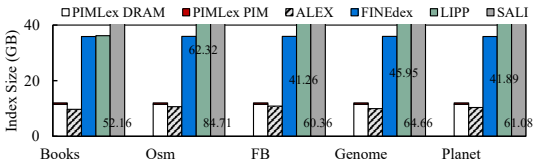
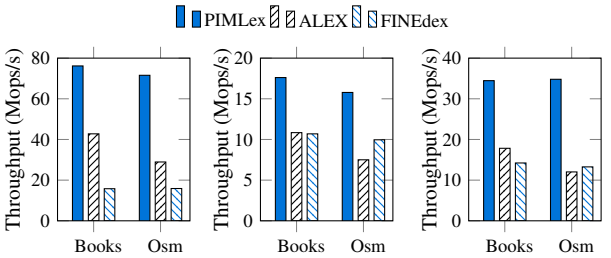


Figure 13: Index Size.

Index Size. Figure 13 demonstrates the memory usage of the compared indexes when they contain 200M key-value pairs. LIPP, FINEdex and SALI require a substantial amount of memory space. In contrast to them, PIMLex demonstrates significant memory efficiency. PIMLex utilizes very little PIM space, mainly due to its decoupled structure, which only places the search layer to PIM. This efficient use of PIM space enables PIMLex to handle larger datasets effectively.

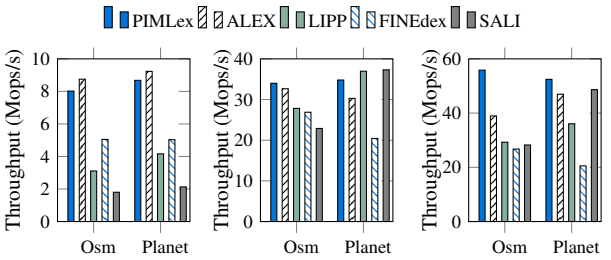
Large Datasets. We evaluate PIMLex with 800 million-scale datasets. The results are presented in Figure 14. The total size of an 800 million-scale dataset is approximately 30GB, while the capacity of the PIM devices on our platform is 32GB. Consequently, an index exclusively using PIM cannot accommodate such large-scale datasets due to the additional space required for other components (e.g., space reserved for future insertions). However, the decoupled structure of PIMLex enables it to efficiently handle these large-scale datasets and exhibit superior performance. PIMLex outperforms ALEX by up to 2.47×, 2.09× and 2.93× for *Get*-only, *Insert*-only and mixed workloads, respectively. For DRAM-based indexes, their performance deteriorates with large datasets because the small-capacity CPU cache cannot accommodate all the hot data. We do not include LIPP and SALI because their index sizes exceed the total memory capacity of the evaluation platform when loading large-scale datasets.

Range Query. We conduct 100 million range query operations with query range 50. Figure 15 exclusively presents



(a) Get (b) Insert (c) Mixed workload

Figure 14: Performance with large-scale datasets.

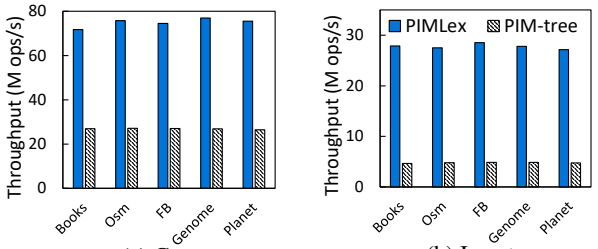


(a) Range Query (b) Update-Heavy (c) Update-Light

Figure 15: PIMLex under other operation types.

results for the *Osm* and *Planet* datasets. Because range query requires large memory accesses on data layer, the advantages brought by PIM side to PIMLex are diminished. PIMLex has close performance to ALEX and achieves advantages over LIPP and SALI. LIPP and SALI have poor scan performance due to the need to traverse nodes on multiple sub-trees.

Update Workloads. We also conduct update-heavy (50% *Get* + 50% *Update*) and update-light (95% *Get* + 5% *Update*) workloads, corresponding to YCSB A and B workloads. Because both *Get* and *Update* follow a Zipfian distribution, PIMLex encounters data contention, diminishing its performance advantage in terms of memory access. But PIMLex still outperforms ALEX, LIPP, FINEdex and SALI by up to 4%, 22%, 26% and 48% under update-heavy workload with *Osm*.



(a) Get. (b) Insert.

Figure 16: Throughput compared with PIM-tree.

4.4 Comparison with PIM-tree

In this section, we compare PIMLex with PIM-tree [26] to highlight the performance advantages of PIMLex compared to prior traditional PIM-based indexes. For this experiment, we utilize 8-byte values since PIM-tree cannot support larger values. We use 100 million-scale datasets because PIM-tree crashes when running on a larger scale in our platform. The results are shown in Figure 16. PIMLex showcases performance superiority over PIM-tree across all workload scenarios, surpassing PIM-tree by up to 3.79× and 6.02× for *Get* and *Insert*, respectively. PIM-tree is engineered to uphold load balance among PIM modules. To achieve this, it fetches index parts from the overloaded PIM modules into main memory and utilizes the host CPU to handle the operations. Although this method can ensure load balancing among PIM modules, it has two flaws. First, executing operations on the host CPU underutilizes the potential of PIM. Second, fetching data from PIM to main memory incurs additional costly CPU-PIM communications [24]. The data transfer bandwidth between CPU and PIM (only 2.5GB/s per DIMM) is significantly lower than that between the CPU and DRAM, resulting in inferior performance. In contrast, the hotness-aware replication mechanism enables PIMLex to effectively handle skewed workloads by leveraging PIM and necessitates less CPU-PIM communication compared to PIM-tree.

4.5 Impact of PIMLex Configurations

Dynamic Workloads. In this experiment, we evaluate the capability of this mechanism with dynamic workloads. We first perform 100 million *Get* operations, and then perform 100 million operations with a changed hotspot distribution. We measure three metrics: the throughput after replication adjustment, the time spent on adjustment and the L_{global} after adjustment. The evaluation results are shown in Table 2. We first discuss the throughput and L_{global}. When using the normal adjustment method, PIMLex executes the complete two-stage load balancing algorithm. After the adjustment, PIMLex achieves a favorable load-balancing state where L_{global} is close to the optimal value of 1. If the quick adjustment method is employed, PIMLex only performs the fine-tuning stage of the load balancing algorithm. In this case, PIMLex adjusts at most Num_fine_tune replicas. As a result, PIMLex is more likely to experience load imbalance. We observe that as Num_fine_tune decreases, L_{global} increases, indicating a higher degree of load imbalance. Consequently, PIMLex suffers from a throughput degradation ranging from 10% to 31% compared to the normal adjustment method. Regarding the adjustment time, the normal adjustment method takes the longest time. This is primarily due to the time required for loading replicas into the PIM modules. On the other hand, the quick adjustment method exhibits a shorter time as fewer replicas are transferred to the PIM modules. By default, PIMLex uses the normal adjustment method because it offers higher throughput and the hotness distribution of real workloads tends to be relatively stable over a short period [4, 8, 53]. However, for extremely dynamic workloads, the quick method should be applied.

Anchor Interval. The default anchor interval is 8. We also

Table 2: Replication adjustment performance. *NA* represents the normal adjustment method. $QA(x)$ represents the quick adjustment method with Num_fine_tune is x. The results are measured under *Osm* dataset.

	NA	QA(32)	QA(16)	QA(8)
Throughput (M ops/s)	75.8	68.8	60.1	53.7
Adjustment time (sec)	0.45	0.31	0.21	0.15
L_{global}	1.14	1.58	1.98	2.54

measure the *Get* performance at different values of anchor interval. The anchor interval in PIMLex has a trade-off. A larger interval leads to more memory accesses and deteriorates performance. When the interval increases from 8 to 32, the *Get* performance decreases by 15%. However, it also means fewer anchor keys and a smaller search layer, which can be advantageous for accommodating larger datasets.

5 Discussions

Generic ability of PIMLex. Although PIMLex is implemented based on UPMEM, its design can also be applied to other Processing-In-Memory (PIM) architectures. First, apart from UPMEM, other PIM typically have smaller capacity [34,37]. Conversely, non-computing memory technologies with larger capacities, such as CXL memory [54] and non-volatile memory (NVM) [13], are continually evolving. In this case, the decoupled structure of PIMLex makes full use of different memory devices to effectively handle large amounts of data. Moreover, other PIM devices generally exhibit robust memory access capabilities but limited computing capabilities. For example, AiM [37] boasts 1 TB/s of bandwidth but only 1 TFLOP/s of computing capability per chip. The model structure of PIMLex adheres to the concept of "trading more memory access for less computation", remains viable for other PIM devices. Additionally, other PIM devices, such as AiM [37] and HBM-PIM [34], also comprise multiple independent modules. The load balancing mechanism of PIMLex is adaptable to these architectures as well.

6 Related Work

PIM Technologies. There are numerous real-world PIM architectures [29, 34, 36, 37, 52] proposed. Samsung [34] proposes a dedicated PIM based on High Bandwidth Memory (HBM) technology, specifically designed to accelerate machine learning. AxDIMM [29, 36] integrates FPGA and DRAM rank in a PIM module, aiming to accelerate database operations and recommendation systems. SK Hynix AiM [37] combines vector processing units with GDDR6 memory banks to accelerate deep learning. In contrast to these specialized architectures, UPMEM stands out as a general-purpose architecture.

Data Processing Applications using PIM. Prior studies [1, 5, 6, 23, 43, 51, 64] explore various applications using PIM. GraphP [64] and GraphQ [67] leverage PIM built with die-stack memories for graph processing. PIM-STM [45] proposes a software transactional memory framework based on PIM architecture. PIMPR [62] uses PIM to accelerate recommendation systems. Lim et al. [43] evaluate the join operation on UPMEM and propose a fast PID-join algorithm with efficient data partition and transfer. PIMDB [6] is an initial PIM-enabled database implemented on UPMEM. Baumstark et al. [5] propose to use PIM to accelerate large table scanning processes. Numerous studies [9, 26, 27, 44] also focus on index design for PIM, presenting various paradigms that integrate traditional indexes such as trie tree and skiplist into PIM. PIMLex is the pioneering PIM-based learned index, addressing specific issues related to learned indexes, such as model stricture optimization. Additionally, many previous works rely on simulated PIM hardware, but PIMLex utilizes real UPMEM devices.

Learned Indexes. In addition to the aforementioned learned indexes, there is other related work [19, 31, 40, 41, 46, 50, 56, 57, 60, 66]. FITing-Tree [19] use linear regression models to replace the leaf nodes of B+-tree. XIndex [56] introduces RCU mechanism to improve concurrency efficiency. NFL [60] builds a learned index after transforming the distribution of data, making more accuracy model predictions. DILI [41] proposes a two-phase bulk loading approach to trade off the index height and the number of nodes. RadixSpline [31] uses interpolation method to construct the models. SIndex [57] focus on optimizing for string keys. Hyper [65] combines both bottom-up and top-down construction approaches to balance the index size and performance. ROLEX [40] applies the idea of learned indexes in disaggregated memory systems. PIMLex introduces a new paradigm by leveraging PIM to overcome the memory wall challenge.

7 Conclusion

This paper presents PIMLex, which preliminarily explores the potential of PIM to address the memory-bound issues of learned indexes. PIMLex utilizes a decoupled structure to minimize PIM capacity usage. Meanwhile, PIMLex optimizes its model structure based on PIM module features. Furthermore, PIMLex employs a hotness-aware replication mechanism to maintain load balancing among PIM modules. Evaluation on real-world PIM hardware demonstrates the superiority of PIMLex over ordinary PIM-based baseline and DRAM-based learned indexes.

Acknowledgments

We sincerely thank our shepherd Yu Hua and the anonymous reviewers. This work was supported in part by the National Science Foundation of China (No. 62272252, 62272253).

References

[1] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. A scalable processing-in-memory accelerator for parallel graph processing. In *2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA)*, pages 105–117, 2015.

[2] Mikkel Møller Andersen and Pinar Tözün. Micro-architectural analysis of a learned index. In *Proceedings of the Fifth International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*, aiDM '22, 2022.

[3] Shaahin Angizi, Zhezhi He, Adnan Siraj Rakin, and Deliang Fan. Cmp-pim: An energy-efficient comparator-based processing-in-memory neural network accelerator. In *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*, pages 1–6, 2018.

[4] Berk Atikoglu, Yuehai Xu, Eitan Frachtenberg, Song Jiang, and Mike Paleczny. Workload analysis of a large-scale key-value store. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE Joint International Conference on Measurement and Modeling of Computer Systems*, SIGMETRICS '12, page 53–64, New York, NY, USA, 2012. Association for Computing Machinery.

[5] Alexander Baumstark, Muhammad Attahir Jibril, and Kai-Uwe Sattler. Accelerating large table scan using processing-in-memory technology. In *Datenbanksysteme für Business, Technologie und Web (BTW 2023), 20. Fachtagung des GI-Fachbereichs „Datenbanken und Informationssysteme" (DBIS), 06.-10, März 2023, Dresden, Germany, Proceedings*, volume P-331 of *LNI*, pages 797–814. Gesellschaft für Informatik e.V., 2023.

[6] Arthur Bernhardt, Andreas Koch, and Ilia Petrov. Pimdb: From main-memory dbms to processing-in-memory dbms-engines on intelligent memories. In *Proceedings of the 19th International Workshop on Data Management on New Hardware*, DaMoN '23, page 44–52, New York, NY, USA, 2023.

[7] Zhichao Cao, Siying Dong, Sagar Vemuri, and David H. C. Du. Characterizing, modeling, and benchmarking rocksdb key-value workloads at facebook. In *Proceedings of the 18th USENIX Conference on File and Storage Technologies*, FAST'20, page 209–224, 2020.

[8] Jiqiang Chen, Liang Chen, Sheng Wang, Guoyun Zhu, Yuanyuan Sun, Huan Liu, and Feifei Li. Hotring: A hotspot-aware in-memory key-value store. In *18th USENIX Conference on File and Storage Technologies (FAST 20)*, pages 239–252, 2020.

[9] Jiwon Choe, Amy Huang, Tali Moreshet, Maurice Herlihy, and R. Iris Bahar. Concurrent data structures with near-data-processing: An architecture-aware implementation. In *The 31st ACM Symposium on Parallelism in Algorithms and Architectures*, SPAA '19, page 297–308, New York, NY, USA, 2019. Association for Computing Machinery.

[10] UPMEM Company. Introduction to UPMEM DIMM. https://www.upmem.com/technology/.

[11] UPMEM Company. UPMEM SDK User Manual. https://sdk.upmem.com/2023.1.0/index.html.

[12] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with ycsb. In *Proceedings of the 1st ACM Symposium on Cloud Computing*, SoCC '10, 2010.

[13] Intel Corporation. Intel Optane DC Persistent Memory. https://www.intel.com/content/www/us/en/products/docs/memory-storage/optane-persistent-memory/overview.html.

[14] Andrew Crotty. Hist-tree: Those who ignore it are doomed to learn. In *11th Conference on Innovative Data Systems Research, CIDR 2021,Virtual Event, January 11-15, 2021, Online Proceedings*. www.cidrdb.org, 2021.

[15] Prangon Das, Purab Ranjan Sutradhar, Mark Indovina, Sai Manoj Pudukotai Dinakarrao, and Amlan Ganguly. Implementation and evaluation of deep neural networks in commercially available processing in memory hardware. In *2022 IEEE 35th International System-on-Chip Conference (SOCC)*, pages 1–6, 2022.

[16] Fabrice Devaux. The true processing in memory accelerator. In *2019 IEEE Hot Chips 31 Symposium (HCS)*, pages 1–24, 2019.

[17] Jialin Ding, Umar Farooq Minhas, Jia Yu, Chi Wang, Jaeyoung Do, Yinan Li, Hantian Zhang, Badrish Chandramouli, Johannes Gehrke, Donald Kossmann, David Lomet, and Tim Kraska. Alex: An updatable adaptive learned index. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, SIGMOD '20, page 969–984, 2020.

[18] Paolo Ferragina and Giorgio Vinciguerra. The pgm-index: A fully-dynamic compressed learned index with provable worst-case bounds. *Proc. VLDB Endow.*, 13(8):1162–1175, apr 2020.

[19] Alex Galakatos, Michael Markovitch, Carsten Binnig, Rodrigo Fonseca, and Tim Kraska. Fiting-tree: A data-aware index structure. In *Proceedings of the 2019 International Conference on Management of Data*, SIGMOD '19, page 1189–1206, 2019.

[20] Mingyu Gao, Jing Pu, Xuan Yang, Mark Horowitz, and Christos Kozyrakis. Tetris: Scalable and efficient neural network acceleration with 3d memory. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '17, page 751–764, New York, NY, USA, 2017. Association for Computing Machinery.

[21] Jiake Ge, Huanchen Zhang, Boyu Shi, Yuanhui Luo, Yunda Guo, Yunpeng Chai, Yuxing Chen, and Anqun Pan. Sali: A scalable adaptive learned index framework based on probability models. *Proc. ACM Manag. Data*, 1(4), December 2023.

[22] Christina Giannoula, Nandita Vijaykumar, Nikela Papadopoulou, Vasileios Karakostas, Ivan Fernandez, Juan Gómez-Luna, Lois Orosa, Nectarios Koziris, Georgios Goumas, and Onur Mutlu. Syncron: Efficient synchronization support for near-data-processing architectures. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 263–276, 2021.

[23] Juan Gómez-Luna, Yuxin Guo, Sylvan Brocard, Julien Legriel, Remy Cimadomo, Geraldo F. Oliveira, Gagandeep Singh, and Onur Mutlu. Machine learning training on a real processing-in-memory system. In *2022 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pages 292–295, 2022.

[24] Juan Gómez-Luna, Izzat El Hajj, Ivan Fernandez, Christina Giannoula, Geraldo F. Oliveira, and Onur Mutlu. Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system. *IEEE Access*, 10:52565–52608, 2022.

[25] Yu Huang, Long Zheng, Pengcheng Yao, Jieshan Zhao, Xiaofei Liao, Hai Jin, and Jingling Xue. A heterogeneous pim hardware-software co-design for energy-efficient graph processing. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 684–695, 2020.

[26] Hongbo Kang, Yiwei Zhao, Guy E. Blelloch, Laxman Dhulipala, Yan Gu, Charles McGuffey, and Phillip B. Gibbons. Pim-tree: A skew-resistant index for processing-in-memory. *Proc. VLDB Endow.*, 16(4):946–958, dec 2022.

[27] Hongbo Kang, Yiwei Zhao, Guy E. Blelloch, Laxman Dhulipala, Yan Gu, Charles McGuffey, and Phillip B. Gibbons. Pim-trie: A skew-resistant trie for processing-in-memory. In *Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures*, SPAA '23, page 1–14, New York, NY, USA, 2023. Association for Computing Machinery.

[28] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S. Lee, Meng Li, Bert Maher, Dheevatsa Mudigere, Maxim Naumov, Martin Schatz, Mikhail Smelyanskiy, Xiaodong Wang, Brandon Reagen, Carole-Jean Wu, Mark Hempstead, and Xuan Zhang. Recnmp: Accelerating personalized recommendation with near-memory processing. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 790–803, 2020.

[29] Liu Ke, Xuan Zhang, Jinin So, Jong-Geon Lee, Shin-Haeng Kang, Sukhan Lee, Songyi Han, YeonGon Cho, Jin Hyun Kim, Yongsuk Kwon, KyungSoo Kim, Jin Jung, Ilkwon Yun, Sung Joo Park, Hyunsun Park, Joonho Song, Jeonghyeon Cho, Kyomin Sohn, Nam Sung Kim, and Hsien-Hsin S. Lee. Near-memory processing in action: Accelerating personalized recommendation with axdimm. *IEEE Micro*, 42(1):116–127, 2022.

[30] Ji-Hoon Kim, Juhyoung Lee, Jinsu Lee, Jaehoon Heo, and Joo-Young Kim. Z-pim: A sparsity-aware processing-in-memory architecture with fully variable weight bit-precision for energy-efficient deep neural networks. *IEEE Journal of Solid-State Circuits*, 56(4):1093–1104, 2021.

[31] Andreas Kipf, Ryan Marcus, Alexander van Renen, Mihail Stoian, Alfons Kemper, Tim Kraska, and Thomas Neumann. Radixspline: A single-pass learned index. In *Proceedings of the Third International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*, aiDM '20, 2020.

[32] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. The case for learned index structures. In *Proceedings of the 2018 International Conference on Management of Data*, SIGMOD '18, page 489–504, 2018.

[33] Yongkee Kwon, Kornijcuk Vladimir, Nahsung Kim, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Guhyun Kim, Byeongju An, Jeongbin Kim, Jaewook Lee, Ilkon Kim, Jaehan Park, Chanwook Park, Yosub Song, Byeongsu Yang, Hyungdeok Lee, Seho Kim, Daehan Kwon, Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, Junyeol Jeon, Myeongjun Lee, Minyoung Shin, Minhwan Shin, Jaekyung Cha, Changson Jung, Kijoon Chang, Chunseok Jeong, Euicheol Lim, Il Park, Junhyun Chun, and Sk Hynix. System architecture and software stack for gddr6-aim. In *2022 IEEE Hot Chips 34 Symposium (HCS)*, pages 1–25, 2022.

[34] Young-Cheon Kwon, Suk Han Lee, Jaehoon Lee, Sang-Hyuk Kwon, Je Min Ryu, Jong-Pil Son, O Seongil, Hak-

Soo Yu, Haesuk Lee, Soo Young Kim, Youngmin Cho, Jin Guk Kim, Jongyoon Choi, Hyun-Sung Shin, Jin Kim, BengSeng Phuah, HyoungMin Kim, Myeong Jun Song, Ahn Choi, Daeho Kim, SooYoung Kim, Eun-Bong Kim, David Wang, Shinhaeng Kang, Yuhwan Ro, Seungwoo Seo, JoonHo Song, Jaeyoun Youn, Kyomin Sohn, and Nam Sung Kim. 25.4 a 20nm 6gb function-in-memory dram, based on hbm2 with a 1.2tflops programmable computing unit using bank-level parallelism, for machine learning applications. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 64, pages 350–352, 2021.

[35] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. Tensordimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '52, page 740–753, New York, NY, USA, 2019. Association for Computing Machinery.

[36] Donghun Lee, Jinin So, Minseon Ahn, Jong-Geon Lee, Jungmin Kim, Jeonghyeon Cho, Oliver Rebholz, Vishnu Charan Thummala, Ravi Shankar JV, Sachin Suresh Upadhya, Mohammed Ibrahim Khan, and Jin Hyun Kim. Improving in-memory database operations with acceleration DIMM (axdimm). In *International Conference on Management of Data, DaMoN 2022, Philadelphia, PA, USA, 13 June 2022*, pages 2:1–2:9. ACM, 2022.

[37] Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, Junyeol Jeon, Nahsung Kim, Yongkee Kwon, Kornijcuk Vladimir, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Jaewook Lee, Donguc Ko, Younggun Jun, Keewon Cho, Ilwoong Kim, Choungki Song, Chunseok Jeong, Daehan Kwon, Jieun Jang, Il Park, Junhyun Chun, and Joohwan Cho. A 1ynm 1.25v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep-learning applications. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 1–3, 2022.

[38] Cong Li, Zhe Zhou, Yang Wang, Fan Yang, Ting Cao, Mao Yang, Yun Liang, and Guangyu Sun. Pim-dl: Expanding the applicability of commodity dram-pims for deep learning via algorithm-system co-optimization. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '24, page 879–896, New York, NY, USA, 2024. Association for Computing Machinery.

[39] Pengfei Li, Yu Hua, Jingnan Jia, and Pengfei Zuo. Finedex: A fine-grained learned index scheme for scalable and concurrent memory systems. *Proc. VLDB Endow.*, 15(2):321–334, feb 2022.

[40] Pengfei Li, Yu Hua, Pengfei Zuo, Zhangyu Chen, and Jiajie Sheng. ROLEX: A scalable RDMA-oriented learned Key-Value store for disaggregated memory systems. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*, pages 99–114, Santa Clara, CA, February 2023. USENIX Association.

[41] Pengfei Li, Hua Lu, Rong Zhu, Bolin Ding, Long Yang, and Gang Pan. Dili: A distribution-driven learned index. *Proc. VLDB Endow.*, 16(9):2212–2224, may 2023.

[42] Wen Li, Ying Wang, Huawei Li, and Xiaowei Li. P3m: a pim-based neural network model protection scheme for deep learning accelerator. In *Proceedings of the 24th Asia and South Pacific Design Automation Conference*, ASPDAC '19, page 633–638, New York, NY, USA, 2019. Association for Computing Machinery.

[43] Chaemin Lim, Suhyun Lee, Jinwoo Choi, Jounghoo Lee, Seongyeon Park, Hanjun Kim, Jinho Lee, and Youngsok Kim. Design and analysis of a processing-in-dimm join algorithm: A case study with upmem dimms. *Proc. ACM Manag. Data*, 1(2), jun 2023.

[44] Zhiyu Liu, Irina Calciu, Maurice Herlihy, and Onur Mutlu. Concurrent data structures for near-memory computing. In *Proceedings of the 29th ACM Symposium on Parallelism in Algorithms and Architectures*, SPAA '17, page 235–245, New York, NY, USA, 2017. Association for Computing Machinery.

[45] André Lopes, Daniel Castro, and Paolo Romano. Pim-stm: Software transactional memory for processing-in-memory systems. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '24, page 897–911, New York, NY, USA, 2024. Association for Computing Machinery.

[46] Baotong Lu, Jialin Ding, Eric Lo, Umar Farooq Minhas, and Tianzheng Wang. Apex: A high-performance learned index on persistent memory. *Proc. VLDB Endow.*, 15(3):597–610, nov 2021.

[47] Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoian, Sanchit Misra, Alfons Kemper, Thomas Neumann, and Tim Kraska. Benchmarking learned indexes. *Proc. VLDB Endow.*, 14(1):1–13, sep 2020.

[48] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungnirun. A modern primer on processing in memory. *CoRR*, abs/2012.03112, 2020.

[49] Lifeng Nai, Ramyad Hadidi, Jaewoong Sim, Hyojong Kim, Pranith Kumar, and Hyesoon Kim. Graphpim: Enabling instruction-level pim offloading in graph computing frameworks. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 457–468, 2017.

[50] Vikram Nathan, Jialin Ding, Mohammad Alizadeh, and Tim Kraska. Learning multi-dimensional indexes. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, SIGMOD '20, page 985–1000, New York, NY, USA, 2020. Association for Computing Machinery.

[51] Joel Nider, Craig Mustard, Andrada Zoltan, John Ramsden, Larry Liu, Jacob Grossbard, Mohammad Dashti, Romaric Jodin, Alexandre Ghiti, Jordi Chauzi, and Alexandra Fedorova. A case study of Processing-in-Memory in off-the-Shelf systems. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 117–130. USENIX Association, July 2021.

[52] Dimin Niu, Shuangchen Li, Yuhao Wang, Wei Han, Zhe Zhang, Yijin Guan, Tianchan Guan, Fei Sun, Fei Xue, Lide Duan, Yuanwei Fang, Hongzhong Zheng, Xiping Jiang, Song Wang, Fengguo Zuo, Yubing Wang, Bing Yu, Qiwei Ren, and Yuan Xie. 184qps/w 64mb/mm23d logic-to-dram hybrid bonding with process-near-memory engine for recommendation system. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 1–3, 2022.

[53] Ziyue Qiu, Juncheng Yang, Juncheng Zhang, Cheng Li, Xiaosong Ma, Qi Chen, Mao Yang, and Yinlong Xu. Frozenhot cache: Rethinking cache management for modern hardware. In *Proceedings of the Eighteenth European Conference on Computer Systems*, EuroSys '23, page 557–573, New York, NY, USA, 2023. Association for Computing Machinery.

[54] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. Demystifying cxl memory with genuine cxl-ready systems and devices. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '23, page 105–121, New York, NY, USA, 2023. Association for Computing Machinery.

[55] Zhaoyan Sun, Xuanhe Zhou, and Guoliang Li. Learned index: A comprehensive experimental evaluation. *Proc. VLDB Endow.*, 16(8):1992–2004, apr 2023.

[56] Chuzhe Tang, Youyun Wang, Zhiyuan Dong, Gansen Hu, Zhaoguo Wang, Minjie Wang, and Haibo Chen. Xindex: A scalable learned index for multicore data storage. In *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '20)*, page 308–320, 2020.

[57] Youyun Wang, Chuzhe Tang, Zhaoguo Wang, and Haibo Chen. Sindex: A scalable learned index for string keys. In *Proceedings of the 11th ACM SIGOPS Asia-Pacific Workshop on Systems*, APSys '20, page 17–24, 2020.

[58] Chaichon Wongkham, Baotong Lu, Chris Liu, Zhicong Zhong, Eric Lo, and Tianzheng Wang. Are updatable learned indexes ready? *Proc. VLDB Endow.*, 15(11):3004–3017, sep 2022.

[59] Jiacheng Wu, Yong Zhang, Shimin Chen, Jin Wang, Yu Chen, and Chunxiao Xing. Updatable learned index with precise positions. *Proc. VLDB Endow.*, 14(8):1276–1288, apr 2021.

[60] Shangyu Wu, Yufei Cui, Jinghuan Yu, Xuan Sun, Tei-Wei Kuo, and Chun Jason Xue. Nfl: Robust learned index via distribution transformation. *Proc. VLDB Endow.*, 15(10):2188–2200, sep 2022.

[61] Juncheng Yang, Yao Yue, and K. V. Rashmi. A large-scale analysis of hundreds of in-memory key-value cache clusters at twitter. *ACM Trans. Storage*, 17(3), aug 2021.

[62] Tao Yang, Hui Ma, Yilong Zhao, Fangxin Liu, Zhezhi He, Xiaoli Sun, and Li Jiang. Pimpr: Pim-based personalized recommendation with heterogeneous memory hierarchy. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–6, 2023.

[63] Huanchen Zhang, David G. Andersen, Andrew Pavlo, Michael Kaminsky, Lin Ma, and Rui Shen. Reducing the storage overhead of main-memory oltp databases with hybrid indexes. In *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, page 1567–1581, 2016.

[64] Mingxing Zhang, Youwei Zhuo, Chao Wang, Mingyu Gao, Yongwei Wu, Kang Chen, Christos Kozyrakis, and Xuehai Qian. Graphp: Reducing communication for pim-based graph processing with efficient data partition. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 544–557, 2018.

[65] Shunkang Zhang, Ji Qi, Xin Yao, and André Brinkmann. Hyper: A high-performance and memory-efficient learned index via hybrid construction. *Proc. ACM Manag. Data*, 2(3), May 2024.

[66] Zhou Zhang, Zhaole Chu, Peiquan Jin, Yongping Luo, Xike Xie, Shouhong Wan, Yun Luo, Xufei Wu, Peng Zou, Chunyang Zheng, Guoan Wu, and Andy Rudoff.

Plin: A persistent learned index for non-volatile memory with high performance and instant recovery. *Proc. VLDB Endow.*, 16(2):243–255, oct 2022.

[67] Youwei Zhuo, Chao Wang, Mingxing Zhang, Rui Wang, Dimin Niu, Yanzhi Wang, and Xuehai Qian. Graphq: Scalable pim-based graph processing. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '52, page 712–725, New York, NY, USA, 2019. Association for Computing Machinery.